



KIMI K3 ประเมินแนวทางการพัฒนาของ Claude, Gemini, ChatGPT

User:

- PPO_V2.pdf
- แนวทางของ Claude (พัฒนา PPO_V2).pdf
- แนวทางของ ChatGPT (พัฒนา PPO_V2).pdf
- แนวทางของ Gemini (พัฒนา PPO_V2).pdf

นี่คือไฟล์โค้ดและผลลัพธ์ของ PPO_V2 โดยผมได้ส่งให้ Claude, ChatGPT, Gemini ประเมินและเสนอแนวทางการพัฒนา ผมอยากให้คุณลองอ่านทั้ง 4 ไฟล์ PDF นี้แล้วช่วย ประเมินความเห็นของคุณว่าคุณเห็นตรงกันกับใครและไม่ตรงกันกับใครบ้าง จากนั้นให้เขียน แนวทางการพัฒนาที่ถูกต้องในระดับ Production หน่อย เพราะ ผมต้องการพัฒนา PPO_V2 ให้สามารถขึ้น Production ได้จริงโดยจุดประสงค์คือสร้าง Alpha ให้เกิดขึ้นจริง ให้ได้ ,มี Sharpe Ratio เกิน 1.5-2 ขึ้นไป และลด maximum drawdown ให้ต่ำที่สุด โดย Agent จะต้องเรียนรู้ที่จะพยายามเอาชนะตลาดให้ได้ในทุกสภาวะ

Kimi:

ผมได้อ่านทั้ง 4 ไฟล์อย่างละเอียดแล้ว ต่อไปนี้คือการวิเคราะห์เชิงลึกของผม พร้อม แนวทาง Production ที่ผมสังเคราะห์ขึ้นมาจากจุดแข็งของทั้ง 3 ฝ่าย แต่ตัดสิ่งที่ over-engineer หรือ unrealistic ออก

1. วิเคราะห์ความเห็น: ผมเห็นด้วยกับใคร และไม่เห็นด้วยกับใคร
จุดที่ทั้ง 3 เห็นพ้องกัน (และผมเห็นด้วย 100%)

ประเด็น	ความเห็นของผม
Reward Function ต้องเปลี่ยน	ตรงกันทั้ง 3 คน — จาก heuristic penalty (log return — dd — cash drag) เป็น risk-adjusted objective ที่ optimize Sharpe ตรงๆ
ต้องขยาย Universe	ตรงกันทั้ง 3 คน — DIA/QQQ/SPY correlate ~0.9 กัน เป็นไปไม่ได้ทางคณิตศาสตร์ที่จะได้ Sharpe 1.5+ ด้วยสินทรัพย์ที่วิ่งไปทิศทางเดียวกัน



ประเด็น	ความเห็นของผม
ต้องมี Temporal Memory	ตรงกันทั้ง 3 คน — MLP แบบ snapshot วันเดียวไม่พอ ตลาดเป็น non-Markovian
ต้องมี Regime Awareness	ตรงกันทั้ง 3 คน — ตลาดไม่ใช่ i.i.d. distribution เดียว

จุดที่ผมเห็นด้วยกับ **Claude** มากที่สุด

- **Differential Sharpe Ratio (Moody & Saffell)** เป็น “high-leverage move” ที่ควรทำก่อนใครเพื่อน — ไม่ต้องหา data ใหม่ ไม่ต้องแก้ architecture แค่เปลี่ยน reward ก็เปลี่ยนพฤติกรรม agent ได้ทันที
- **ตัด cash-drag penalty แบบ hand-tuned ออก** — ปัญหาหลักของ V2 คือ agent ถูกสอนให้กลัวตลาดเกินไป (fold 5 ถือ cash 78%) การมี penalty 4-5 ตัวที่ interact กันแบบคาดเดายาก ทำให้ reward surface ไม่ convex
- **Evaluation ต้องเข้มข้นกว่านี้** — Sharpe SD 0.59 ในขณะที่ mean 0.47 แปลว่าไม่เสถียรข้าม regime เลย ต้องมี bootstrap CI และ stress-test period แยก

จุดที่ผมเห็นด้วยกับ **ChatGPT** มากที่สุด

- **แนวคิดแยกหน้าที่ (Alpha vs Risk vs Allocation)** มีเหตุผลทาง psychologically — มนุษย์ยังแยก trader / risk manager / PM เลย การบังคับให้ network เดียวเรียนทั้งหมดพร้อมกัน ทำให้ gradient สับสน
- **Market Regime Encoder** เป็นสิ่งจำเป็น — แต่ผมคิดว่าไม่ต้องถึง Transformer ขนาดนั้น LSTM/GRU บน macro time series ก็พอ
- **Curriculum Learning** ดีสำหรับ stability — เริ่มจากตลาดง่าย ค่อยๆ เพิ่ม crisis

จุดที่ผมเห็นด้วยกับ **Gemini** มากที่สุด

- **Two-Tier Architecture (Alpha + Vol Targeting Overlay)** เป็น approach ที่ implement ได้จริงที่สุด — ไม่ต้องรอให้ RL เรียน risk management เอง แต่ใช้ deterministic math คม drawdown ไว้ก่อน
- **Benchmark-Relative Excess Return** ต้องเป็นหัวใจของ reward — ถ้าตลาดบวก 2% แต่ agent ถือ cash ได้ 0% ต้องโดนลงโทษหนัก (ไม่ใช่แค่ไม่ได้รางวัล)



- **Volatility Targeting เป็น deterministic overlay ที่ proven ใน industry** — AQR, Bridgewater, Two Sigma ใช้หมด
-

จุดที่ผม **ไม่เห็นด้วย** กับ ChatGPT

ข้อเสนอ

ทำไมผมไม่เห็นด้วย

**Multi-Agent 3 ตัวแยก
ขัดเกินไป**

ซับซ้อนและ training instability สูงมาก สำหรับ
ข้อมูล ~5,000 วัน การ train 3 policy networks
พร้อมกัน มีแนวโน้ม overfit และ non-convergence
สูง

DreamerV3

Overkill สำหรับขนาดนี้ ต้องการ data เยอะกว่านี้
มากสำหรับ world model training time นานเกินไป
สำหรับ iteration speed ที่คุณต้องการ

**เพิ่ม Universe เป็น
30-50 assets ในครั้ง
เดียว**

Curse of dimensionality จะทำลาย sample
efficiency ของ PPO ทันที ควรเริ่มจาก 5-6 assets
ที่มี economic rationale ก่อน

จุดที่ผม **ไม่เห็นด้วย** กับ Gemini

ข้อเสนอ

ทำไมผมไม่เห็นด้วย

**Dirichlet Distribution
แทน Softmax**

ซับซ้อนเกินไปสำหรับผลลัพธ์ที่ไม่ต่างจาก Softmax
+ No-Trade Band เท่าที่ควร การ implement
Dirichlet บน continuous action space ของ PPO
มี numerical stability issues

**Transformer Encoder
บน macro features**

อาจ overfit ถ้า data น้อย (มีแค่ ~6,000 วัน)
Transformer ต้องการ data scale ใหญ่มากกว่านี้มาก
ควรใช้ LSTM/GRU ก่อน

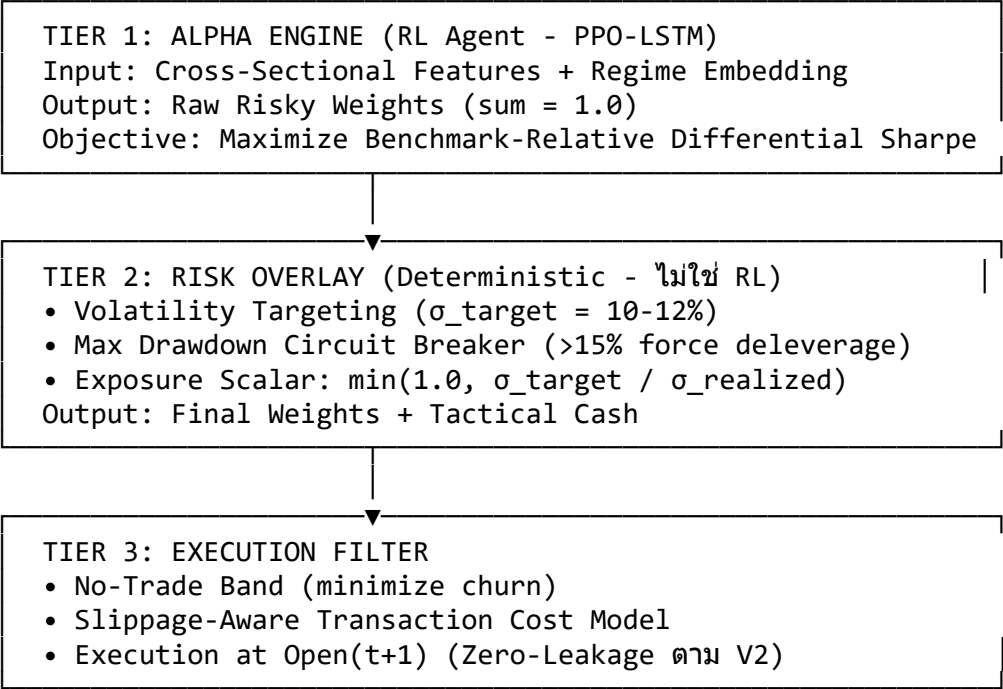
จุดที่ผม **ไม่เห็นด้วย** กับ Claude

- ไม่มีจุดที่ไม่เห็นด้วยอย่างมีนัยสำคัญ แต่ผมคิดว่า **Claude ไม่ได้เน้นเรื่อง
Production Pipeline และ Hard Risk Constraint พอ** — การมี reward ที่ดี
ไม่พอ ต้องมี circuit breaker ที่เป็นอิสระจาก policy ด้วย
-



2. แนวทาง Production ที่ผมสังเคราะห์: “Hierarchical Alpha-Risk Architecture”

ผมรวมสิ่งที่ดีที่สุดจากทั้ง 3 ฝ่าย แล้วตัดสิ่งที่ over-engineer ออก ได้สถาปัตยกรรม 3 Tier ดังนี้:



สาเหตุที่ต้องแยกเป็น 3 Tier

Tier	ทำไมต้องแยก
Alpha Engine	RL ควรทำสิ่งที่ RL ถนัด — คือ pattern recognition บน high-dimensional features การจับ relative strength ระหว่าง assets เป็น non-linear problem ที่ RL มี edge
Risk Overlay	Risk management เป็น convex optimization problem ที่ math แก้ได้ตรงๆ ไม่ควรให้ RL เดาเอง (จะได้ policy ที่ inconsistent)
Execution Filter	เป็น operational layer ที่ต้อง robust ไม่ขึ้นกับ policy distribution

2.1 Reward Function ใหม่ (สิ่งสำคัญที่สุด)

```
# Reward(t) = Differential_Sharpe(t) + α * Excess_Return(t) - β *
Downside_Penalty(t) - γ * Turnover_Cost(t)

# 1. Differential Sharpe (Moody & Saffell, 1998)
# ประมาณ marginal contribution ต่อ Sharpe ในแต่ละ step
D_t = (B_{t-1} * ΔA_t - 0.5 * A_{t-1} * ΔB_t) / (B_{t-1} -
A_{t-1}^2)^{3/2}

# 2. Benchmark-Relative Excess Return
# r_benchmark = Equal-Weight Open-to-Open return (เหมือนที่ V2 ทำ)
excess = r_agent - r_eqw

# 3. Downside Penalty (Sortino-style)
# ลงโทษเฉพาะ volatility ผั่งขาด
downside = λ * min(0, r_agent)^2

# 4. Turnover Cost
# ไม่ใช่แค่ fixed bps แต่ขึ้นกับ regime (VIX สูง = slippage สูง)
turnover_cost = turnover * (base_cost + slippage_factor * vix_level)
```

ทำไมต้องเป็นแบบนี้: - V2 ใช้ $\log(1+return)*100 - dd_penalty - cash_drag$ เป็น heuristic ที่ไม่ได้ optimize อะไรตรงๆ - Differential Sharpe ทำให้ agent รู้ว่า “กำไรวันนี้ ดีหรือไม่ดี” ขึ้นอยู่กับว่ามันช่วย Sharpe รวมยังไหม - Excess Return บังคับให้ agent ไม่ดกกรเวลาลดลาบวก (ปัญหาใหญ่ของ V2)

2.2 Universe ที่เหมาะสม (Phase 1)

อย่าเพิ่งไป 30-50 assets ให้เริ่มจาก **6 assets** ที่มี **economic rationale** ชัดเจน:

Asset	บทบาท	Correlation กับ Equity
SPY	Core equity beta	1.0
QQQ	Growth tilt	0.95
DIA	Value/Dividend tilt	0.92
TLT	Long-duration Treasury	-0.3 to -0.5 (risk-off)
IEF	Intermediate Treasury	-0.2 to -0.4 (risk-off)
GLD	Gold / Inflation hedge	0.0 to 0.2 (stagflation alpha)



ทำไมต้องมี TLT + IEF: เวลาตลาด equity ลง เงินสถาบันไหลเข้าพันธบัตร ถ้า agent มี TLT มันจะมี alpha จาก duration exposure แทนที่จะหนีไป cash เปล่าๆ

ทำไมต้องมี GLD: ปี 2022 เป็นตัวอย่าง classic stagflation (หุ้นลง + เงินเฟ้อสูง) ทองคำเป็นสินทรัพย์เดียวที่บวกได้

2.3 Feature Engineering ระดับ Production

ผมแบ่งเป็น 3 กลุ่ม:

A. Cross-Sectional Relative Strength (สิ่งที่ V2 ขาดหายไป)

1. Price Ratio Momentum (Leading indicator ของ rotation)

```
feat["QQQ_SPY_ratio"] = price["QQQ"] / price["SPY"]  
feat["SPY_TLT_ratio"] = price["SPY"] / price["TLT"]
```

2. Relative RSI (ใครแข็งแรงแรงกว่ากันในกลุ่ม)

```
rsi_mean = rsi_df.mean(axis=1)  
feat["RSI_rel_QQQ"] = rsi["QQQ"] - rsi_mean
```

3. Cross-Sectional Momentum Rank (percentile)

```
returns_20d = df.pct_change(20)  
feat["mom_rank"] = returns_20d.rank(pct=True) # 0.0 = อ่อนที่สุด, 1.0 =  
แกร่งที่สุด
```

ทำไมสำคัญ: เมื่อ equity correlate สูง การหา alpha ต้องมาจาก “เลือกตัวที่แกร่งกว่าในกลุ่ม” (relative value) ไม่ใช่ “เดาทิศทางตลาด” (absolute direction)

B. Regime Signals (Macro Pressure)

1. VIX Regime

```
feat["vix_level"] = np.log(vix)  
feat["vix_term"] = vix / vix3m # <1.0 = backwardation (stress)
```

2. Yield Curve (Recession predictor)

```
feat["yc_10y2y"] = yield_10y - yield_2y  
feat["yc_10y3m"] = yield_10y - yield_3m
```

3. Fed Liquidity (Net Liquidity Index)

```
feat["fed_liquidity"] = fed_balance_sheet - tga - reverse_repo
```

4. Credit Stress

```
feat["credit_spread"] = hy_yield - ig_yield
```

5. Market Breadth

```
feat["advance_decline"] = adv_issues - dec_issues
```



C. Temporal Memory (LSTM Input)

Frame stack 60-120 วันย้อนหลัง
Observation ไม่ใช่แค่ features วันนี้ แต่เป็น sequence [t-60:t]
บวกกับ current weights (obs_weights ตาม V2)

2.4 Algorithm: PPO-LSTM + Benchmark-Relative Objective

```
from sb3_contrib import RecurrentPPO

policy_kwargs = dict(
    lstm_hidden_size=128,
    n_lstm_layers=2,
    net_arch=dict(pi=[128, 128], vf=[128, 128]),
    activation_fn=nn.Tanh,
)

model = RecurrentPPO(
    "MlpLstmPolicy",
    env,
    learning_rate=3e-4,
    n_steps=2048,
    batch_size=256,
    n_epochs=10,
    gamma=0.99,
    gae_lambda=0.95,
    policy_kwargs=policy_kwargs,
    verbose=1,
)
```

ทำไมต้อง LSTM: - ตลาดมี memory: trend, cycle, regime shift - EMA ratio ที่ V2 ใช้
 มี memory ในตัวบ้าง แต่เป็น fixed-weight memory - LSTM เรียนเองว่าควรจำอดีตย้อน
 หลังกี่วัน และให้น้ำหนักยังไง

2.5 Risk Constraints (Hard Constraints ที่เป็นอิสระจาก Policy)

อย่าหวังให้ reward function คุมทุกอย่าง ต้องมี **guard rails**:

```
class RiskOverlay:
    def __init__(self, target_vol=0.10, max_dd=0.15):
        self.target_vol_daily = target_vol / np.sqrt(252)
        self.max_dd = max_dd

    def apply(self, raw_weights, realized_vol, current_dd):
        # 1. Volatility Targeting
        scalar = min(1.0, self.target_vol_daily / max(realized_vol,
1e-4))
```



```
risky_weights = raw_weights * scalar
cash_weight = 1.0 - risky_weights.sum()

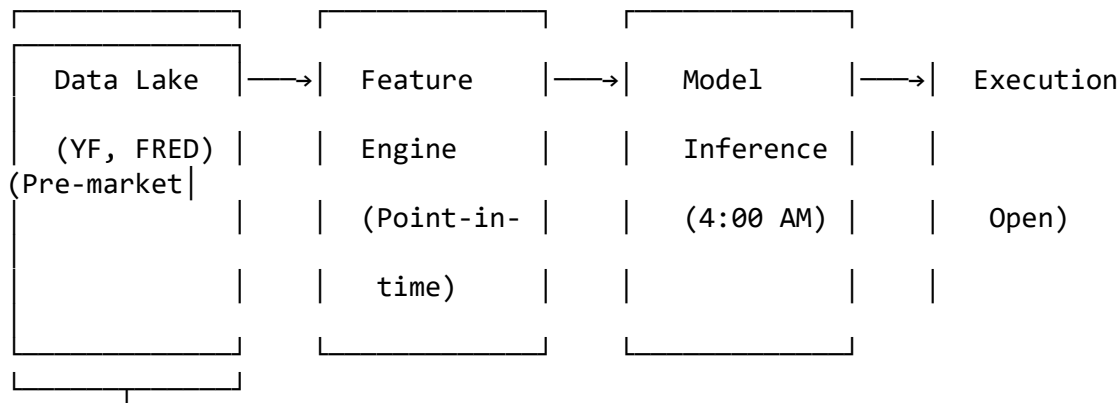
# 2. Max Drawdown Circuit Breaker
if current_dd > self.max_dd:
    cash_weight = 1.0 # Force 100% cash
    risky_weights *= 0.0

return np.concatenate([[cash_weight], risky_weights])
```

2.6 Validation Framework ที่เข้มข้นกว่า V2

เทคนิค	ทำไมต้องมี	ใครเสนอ
Purged	แก้ lookahead bias จาก	ผม + Gemini
Walk-Forward	overlapping labels	
Combinatorial	Test path dependency	Gemini
Purged CV (CPCV)	หลายเส้นทาง	
Deflated Sharpe	ปรับ Sharpe ตามจำนวน	Gemini
Ratio (DSR)	trials ที่ทดสอบ	
Bootstrap CI on	ดูว่า AI ชนะ EqW อย่างมีนัย	Claude
Sharpe Diff	สำคัญหรือแค่ noise	
Cost Sensitivity (15→35bps)	ดูว่า edge ยังอยู่ถ้า slippage แพงกว่าคาด	Gemini
Dedicated Stress Periods	2008Q4, 2020Q1, 2022 ต้องผ่านแยก	Claude + ChatGPT

2.7 Production Pipeline (สิ่งที่ทั้ง 3 คนยังไม่ได้เน้นมากพอ)



MONITORING LAYER (ต้องมีไม่ว่า Sharpe จะดีแค่ไหน)

- Live vs Backtest Tracking (ตรวจ data drift)
- Slippage Attribution (แยกว่า edge หายเพราะ model หรือ execution)
- Regime Detection Alert (ถ้า regime เปลี่ยน ให้ human intervene)
- Hard Stop-Loss (อิสระจาก RL policy เสมอ)

3. Roadmap สู่ Production (เรียงลำดับความสำคัญ)

Phase 1: Foundation (เดือน 1-2) — **Impact สูงสุด ลงทุนต่ำสุด**

- ☐ เปลี่ยน Reward → Differential Sharpe + Excess Return + Downside Penalty
- ☐ ขยาย Universe → [SPY, QQQ, DIA, TLT, IEF, GLD]
- ☐ Implement Two-Tier (Alpha + Vol Targeting)
- ☐ เปลี่ยนเป็น PPO-LSTM (sb3-contrib)
- **เป้าหมาย:** Sharpe > 0.8, ลด cash holding ให้ < 30% ในขาขึ้น

Phase 2: Feature & Regime (เดือน 2-3)

- ☐ Cross-sectional relative strength features
- ☐ Regime encoder (LSTM บน macro time series)
- ☐ Enhanced validation (CPCV + DSR)
- **เป้าหมาย:** Sharpe > 1.0, ชนะ EqW อย่างมีนัยสำคัญ



Phase 3: Risk & Constraints (เดือน 3-4)

- ☐ Hard constraints (MaxDD circuit breaker, position limits)
- ☐ Slippage-aware transaction cost model
- ☐ Stress test 2008/2020/2022
- **เป้าหมาย:** Sharpe > 1.2, MaxDD < 15%

Phase 4: Production (เดือน 4-6)

- ☐ Paper trading 3-6 เดือน
- ☐ Monitoring pipeline (drift detection, attribution)
- ☐ Live deployment ด้วย position sizing conservative (50% ของ backtest optimal)
- **เป้าหมาย:** Sharpe 1.5-2.0 ใน live (realistic target หลัง slippage)

4. สรุปสิ่งที่สำคัญที่สุด (ถ้าทำอะไรไม่ได้ จ้างสิ่งนี้)

#	หลักการ	ทำไมถึง make-or-break
1	Reward ต้อง optimize Sharpe ตรงๆ	V2 ใช้ heuristic ทำให้ agent เรียนผิด — ถือเงินสดเยอะเกิน
2	ต้องมี Benchmark-relative objective	ถ้าตลาดบวก 2% แต่ agent ได้ 0% ต้องโดนลงโทษหนัก (ไม่ใช่แค่ไม่ได้รางวัล)
3	Risk ต้องเป็น deterministic overlay	อย่าปล่อยให้ RL เรียน risk management เอง มันจะ overfit
4	ต้องขยาย universe มี diversifier จริง	ด้วย equity-only เป็นไปไม่ได้ทางคณิตศาสตร์ที่จะได้ Sharpe 1.5+
5	Validation ต้องเข้มกว่านี้มาก	Sharpe SD 0.59 แปลว่า overfit fold ต้องมี CPCV + DSR



- # หลักการ ทำไมถึง make-or-break
- 6 **Production ต้อง** ไม่ควรฝากความเสี่ยงทั้งหมดไว้กับ learned policy
 มี hard
 constraints +
 monitoring

ตอนนี้ผมจะสร้าง **Skeleton Code** และ **Architecture Diagram** ให้คุณดาวน์โหลดไป
ต่อยอดได้เลย: —

สรุป: สิ่งที่คุณต้องทำ **ทันที** (Priority Order)

จากผลลัพธ์ V2 ที่ agent ถือเงินสด 37-78% และแพ้ EqW ทุก fold ปัญหาไม่ใช่ที่ PPO
algorithm แต่อยู่ที่ **สิ่งที่คุณสอนให้มันเรียน (Reward Function)** กับ **เครื่องมือที่คุณ
ให้มันใช้ (Universe)**

ทำสิ่งนี้ก่อน (Week 1-2) — เปลี่ยนผลลัพธ์ทันที:

1. **เปลี่ยน Reward** จาก $\log(1+r) - \text{dd_penalty} - \text{cash_drag}$ เป็น
Differential Sharpe + Excess Return + Downside Penalty
2. **ขยาย Universe** เป็น [SPY, QQQ, DIA, TLT, IEF, GLD] — ให้ agent มี “ทางเลือก” นอกจาก cash เวลาตลาดลง
3. **ใช้ Two-Tier Architecture** — ให้ RL หา alpha อย่างเดียว แล้วใช้ math คำนวณ
risk (vol targeting + circuit breaker)

อย่าทำสิ่งนี้ก่อน (over-engineer):

- ❌ อย่าเพิ่มไป Multi-Agent 3 ตัว (training unstable)
- ❌ อย่าเพิ่มใช้ DreamerV3 (overkill)
- ❌ อย่าเพิ่มขยายเป็น 30-50 assets (curse of dimensionality)



Target ที่ Realistic:

Phase	Timeline	Sharpe Target	MaxDD Target	สถานะ
V2 (Current)	-	0.47	-14%	แพ้ EqW ทุก fold
V3 Phase 1	Month 1-2	> 0.8	< 15%	ชนะ EqW บาง fold
V3 Phase 2	Month 2-3	> 1.0	< 15%	ชนะ EqW อย่างมี นัย สำคัญ
V3 Phase 3	Month 3-4	> 1.2	< 12%	ผ่าน CPCV + DSR
V3 Production	Month 4-6	1.5-2.0	< 10%	Live after paper trading

ไฟล์ที่คุณสามารถดาวน์โหลดได้:

1. Architecture Diagram — แผนผังสถาปัตยกรรม 3 Tier ที่คุณสามารถนำไปอธิบายให้ทีมหรือ investor ฟังได้: [Production Architecture Diagram](#)

2. Production Skeleton Code — โครงสร้าง Python สำหรับ PPO_V3 พร้อม Differential Sharpe, Risk Overlay, PPO-LSTM, และ Production Inference Pipeline: [PPO_V3 Production Skeleton](#)

โค้ดนี้เป็น **skeleton** ที่คุณต้องเติมส่วน process_features_for_slice() และ data pipeline จาก V2 เข้าไป แต่สถาปัตยกรรมหลัก (Reward, Risk Overlay, LSTM Policy, Validation) พร้อมใช้แล้ว



คำถามสุดท้ายจากผม:

คุณต้องการให้ผม **implement** ส่วนไหนให้เต็มรูปแบบ ก่อน? ผมแนะนำให้เริ่มจาก **Reward Function + Risk Overlay + Environment** ตัวอย่าง ที่รันได้จริงบน synthetic data ก่อน เพื่อให้คุณเห็นว่า agent เปลี่ยนพฤติกรรมยังไงหลังจากเปลี่ยน reward แล้ว หรือคุณอยากให้ผมลงลึกเรื่อง **Feature Engineering (Cross-Sectional)** ก่อน?

User:

คุณสามารถเอา ข้อความ, ตาราง, โค้ด ทั้งหมดที่คุณได้ generate ออกมา รวมเป็นเอกสาร แล้ว export เป็นไฟล์ pdf 1 ไฟล์ ได้ไหม?

